

Introduction to Physics and Colliders

Learning Objectives

- Understand the basics of physics simulation in Unity.
- Learn about colliders and their role in collision detection.
- Grasp how to implement basic physics interactions in Unity.

Importance of Physics in Games

1. Realism Enhancement:

- **Simulation of Real-World Behavior:** Physics engines replicate real-world physics, making objects interact realistically with the environment (gravity, friction, collisions, etc.).
- **Environmental Interactions:** Realistic physics create immersive worlds where objects behave as expected, enhancing the player's sense of immersion and believability (Collisions and Interactions, Dynamic Materials).

2. Improving Interactivity:

- **Dynamic Interactions:** Physics engines enable dynamic interactions between game elements, allowing for emergent gameplay and unscripted events.
- **Player Agency:** Physics-based mechanics give players more control and freedom, enabling creative problem-solving and varied gameplay approaches.

Unity's Built-in Physics Engine

Unity helps you simulate physics in your Project to ensure that the objects correctly accelerate and respond to collisions, gravity, and various other forces. Unity provides different physics engine implementations which you can use according to your Project needs: 3D, 2D, object-oriented, or data-oriented. [8]

For developers working with game engines, physics engines provide an accompanying way of simulating real-world responses for objects in games. In Unity, the game engine uses Nvidia's PhysX engine, a popular and highly accurate commercial physics engine. [10]

Built-in physics engines for object-oriented projects

If your project is object-oriented, use the Unity's built-in physics engine that corresponds to your needs:

- [Built-in 3D physics](#) (Nvidia PhysX engine integration)
- [Built-in 2D physics](#) (Box2D engine integration)

2D physics

Unity has a separate physics system to handle 2D physics so you can make use of optimizations only available with 2D. The components correspond to the standard 3D physics components such as Rigidbody, Box Collider and Hinge Joint, but with 2D appended to the name.

Rigidbody 2D: Elaborating on its purpose in determining an object's physical behavior, including movement, rotation, and responses to forces.

Collider 2D: Defining the boundary and shape of an object for collision detection, discussing various collider types (e.g., BoxCollider2D, CircleCollider2D).

Introduction to Rigidbody 2D

You can attach a Rigidbody 2D component to a GameObject to control it with the physics system. The Rigidbody 2D shares similar properties with its standard Rigidbody counterpart, but it's adapted to 2D development.

Unity requires a method for the physics system to communicate this movement of colliders back to the Transform components. This movement and connection with colliders is what a Rigidbody 2D component is for. The Rigidbody 2D component overrides the Transform component and updates it to the position and/or rotation it defines instead.

Rigidbody 2D body types

There are three options for Body Type which define the behavior of the Rigidbody 2D. Any Collider 2D attached to that Rigidbody 2D inherits the Rigidbody 2D's Body Type as well.

Rigidbody 2D types	Description
Dynamic	Use the Dynamic Body Type to design your Rigidbody 2D to move under simulation.
Kinematic	Use the Kinematic Body Type to design your Rigidbody 2D to move under simulation only with explicit user control.
Static	Use the Static Body Type to design your Rigidbody 2D to not move under simulation.

Rigidbody 2D properties: Simulated

The Simulated property is common to all available Body Types. Use this property to start (enabled) or stop (disabled) a Rigidbody 2D and any attached Collider 2Ds and Joint 2Ds from interacting with the 2D physics simulation.

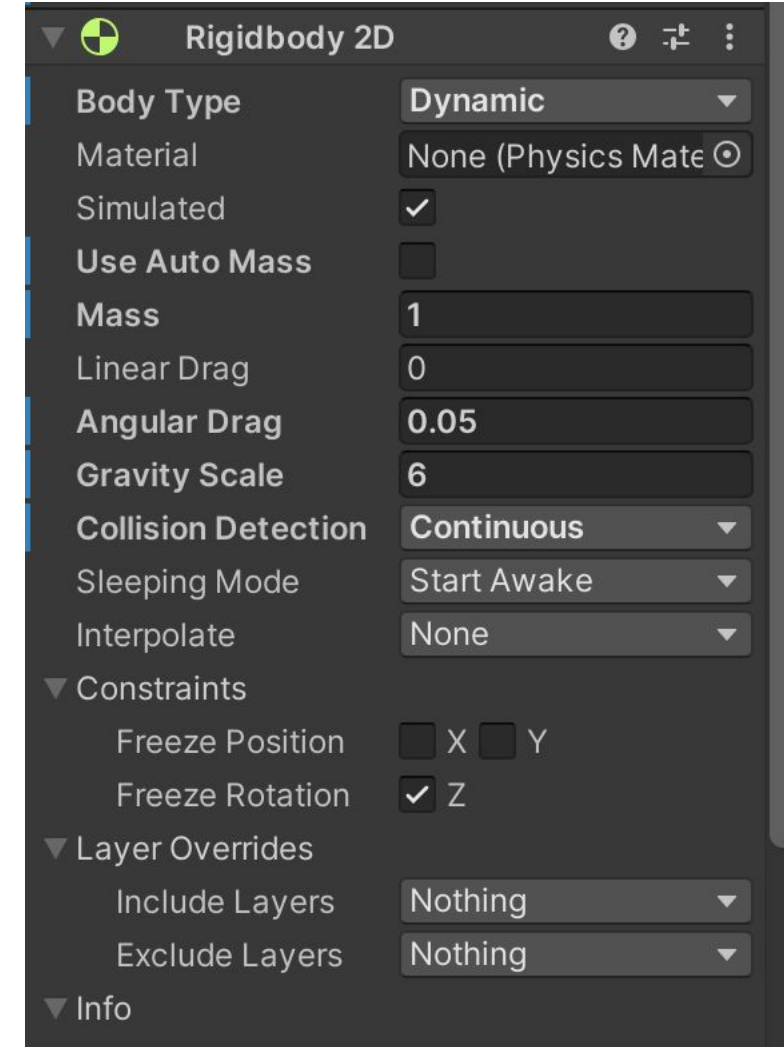
When you enable the Simulated property, the following occurs:

- The Rigidbody 2D moves via the simulation (gravity and physics forces are applied).
- Any attached Collider 2Ds continue creating new contacts and continuously reevaluate contacts.
- Any attached Joint 2Ds are simulated and constrain the attached Rigidbody 2D.
- All internal physics objects for Rigidbody 2D, Collider 2D, and Joint 2D stay in memory.

Rigidbody 2D

Rigidbody 2D properties:

- **Mass:** Impact on the object's inertia and responsiveness to forces.
- **Drag:** Effects on the object's speed reduction in a physical environment.
- **Angular Drag:** Control over rotation speed dampening.
- **Gravity Scale:** Adjusting the influence of gravity on the object.



Collider 2D

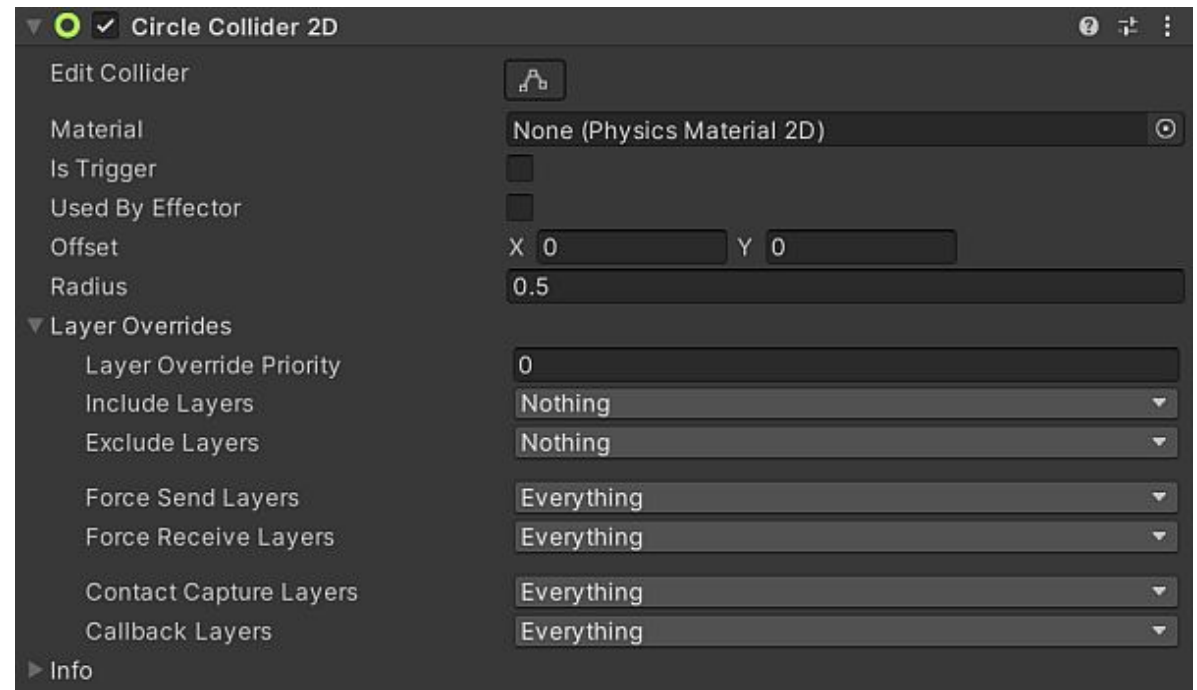
Collider 2D components define the *shape of a 2D GameObject* for the purposes of physical collisions. A collider, which is invisible, doesn't need to be the exact same shape as the GameObject's mesh; a rough approximation is often more efficient, with no noticeable difference in gameplay.

Collider 2D types

Topic	Description
<u>Circle Collider 2D</u>	Use Circle Collider 2D for circular collision areas.
<u>Box Collider 2D</u>	Use <u>Box Collider</u> 2D for square and rectangle collision areas.
<u>Polygon Collider 2D</u>	Use Polygon Collider 2D for freeform collision areas.
<u>Edge Collider 2D</u>	Use Edge Collider 2D freeform collision areas and areas which aren't fully enclosed (such as rounded convex corners).
<u>Capsule Collider 2D</u>	Use <u>Capsule Collider</u> 2D for circular or lozenge-shaped collision areas.
<u>Composite Collider 2D</u>	Use Composite Collider 2D for merging Box Collider 2Ds and Polygon Collider 2Ds.
<u>Custom Collider 2D</u>	Use Custom Collider 2D for collision areas with customizable geometry.

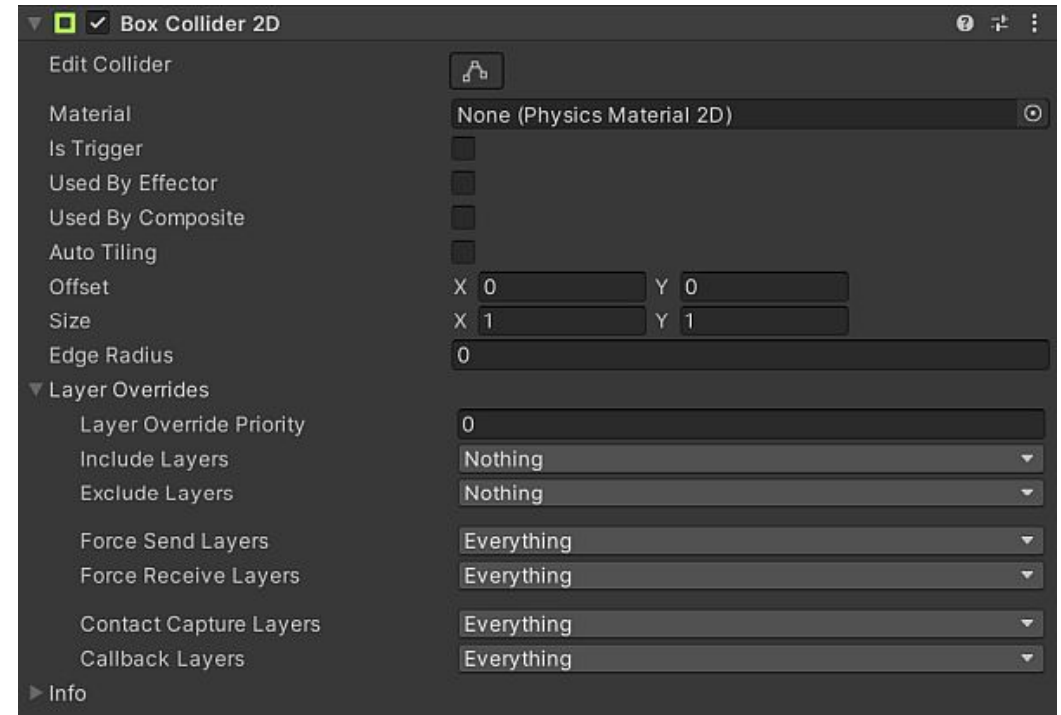
Circle Collider 2D

The Circle Collider 2D is a collider that interacts with the 2D physics system for approximate collision detection. The Circle Collider 2D, as the name suggests, is in circular shape with a defined position and radius within the local coordinate space of a Sprite.



Box Collider 2D

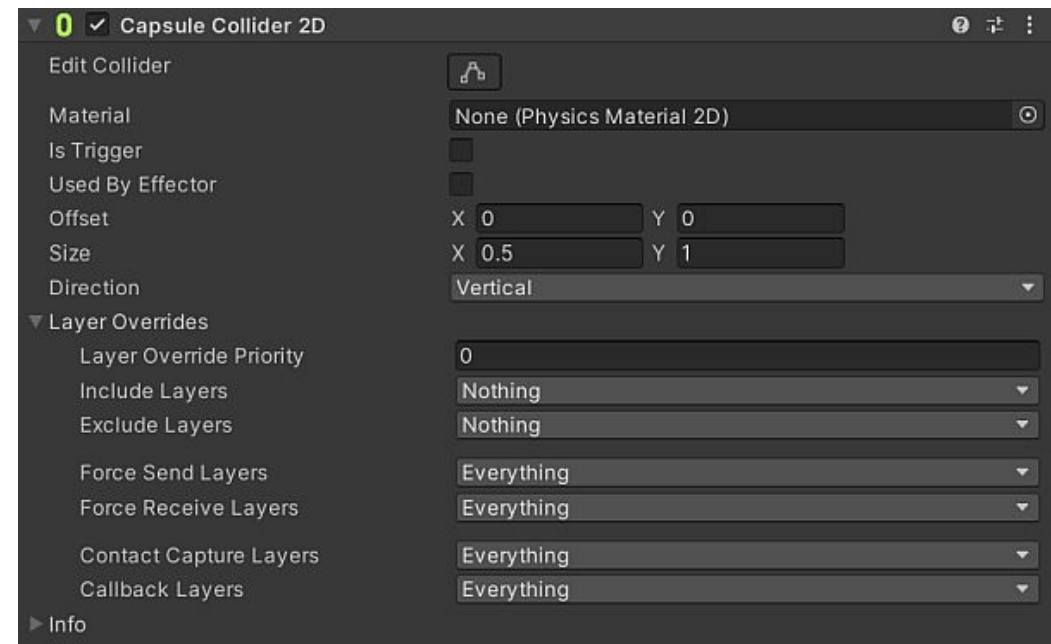
The Box Collider 2D is a Collider that interacts with the 2D physics system. It is a rectangle in shape with a defined position, width and height in the local coordinate space of a Sprite. Note: The selection rectangle is axis-aligned, with its edges parallel to the X or Y axes of local space.



Capsule Collider 2D

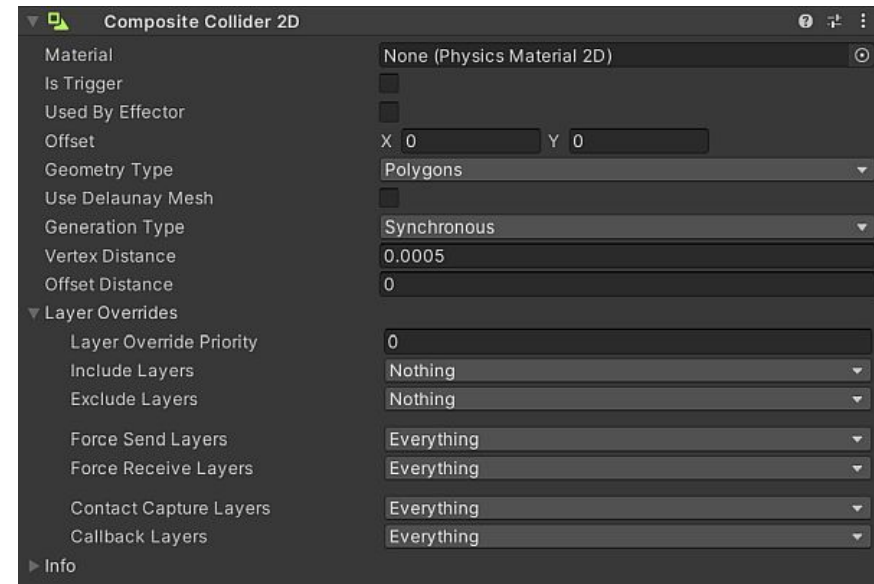
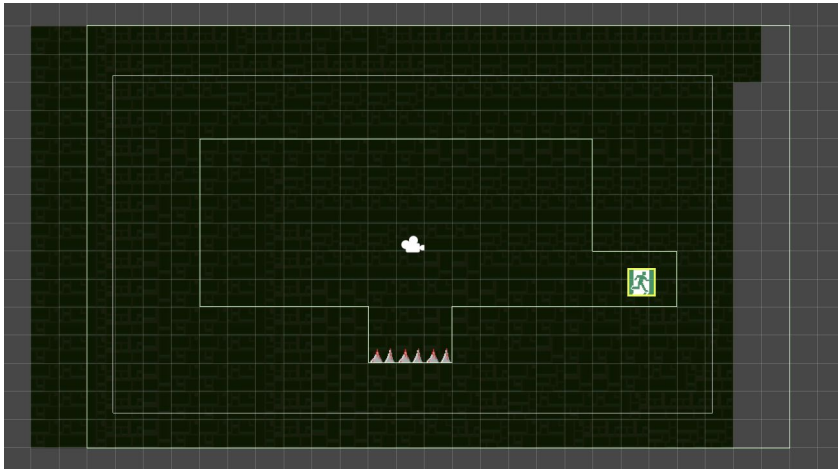
The Capsule Collider 2D is a Collider that interacts with the 2D physics system.

Note: The capsule shape of the Collider is considered solid and not hollow, which means any other Collider 2Ds that are inside the Capsule Collider 2D are considered to be in contact with the Collider and are forced out of it over time.



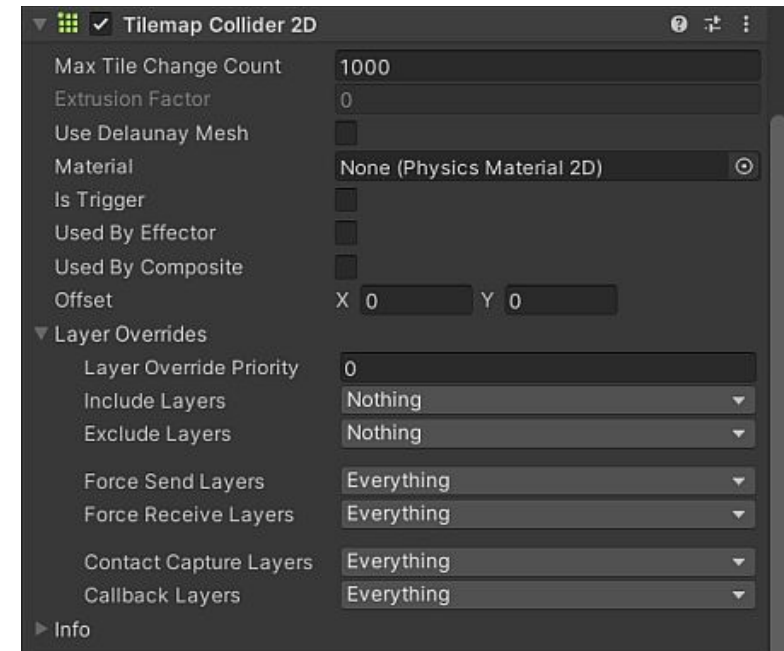
Composite Collider 2D

The Composite Collider 2D component is a Collider that interacts with the 2D physics system. Unlike most Colliders, it does not define an inherent shape. Instead, it ***merges the shapes of any Box Collider 2D or Polygon Collider 2D that you set it up to use***. The Composite Collider 2D uses the vertices (geometry) from any of these Colliders, and merges them together into new geometry controlled by the Composite Collider 2D itself.



Tilemap Collider 2D

The Tilemap Collider 2D component generates Collider shapes for Tiles on a Tilemap component on the same GameObject. When you add or remove Tiles on the Tilemap component, the Tilemap Collider 2D updates the Collider shapes during LateUpdate. It batches multiple Tile changes together to ensure minimal impact on performance.



Tilemap and Composite Colliders

You can use the Tilemap Collider 2D component together with the Composite Collider 2D component. When you add both components to the same Tilemap, Unity composites the Collider shapes of neighboring Tiles together. This smoothens the corners and edges between Collider shapes in neighboring Tiles.

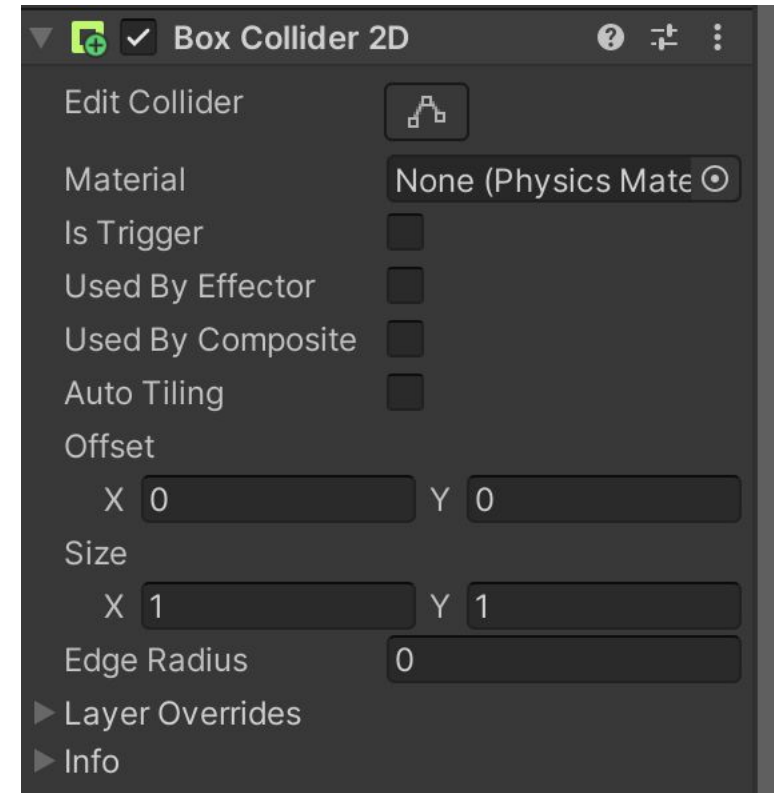
Using both components together reduces the number of individual Collider shapes involved in a physics update, which reduces the amount of calculations required, and minimizes the impact on performance.

Collider 2D and Interaction

Distinction between collision and trigger events using Collider 2D components:

OnCollisionEnter2D: Triggered when two colliders interact, providing opportunities for physical responses or actions.

OnTriggerEnter2D: Utilizing triggers for interaction without physically colliding.



Collision Detection Basics

Collision: Identifying when two or more objects intersect in a game environment.

- Intersection Recognition: Detects when game objects meet.
- Event Triggering: Fundamental for triggering actions based on collisions.

Example: Two game characters colliding, triggering a health deduction event.

Handling Collisions - OnCollisionEnter Method

OnCollisionEnter Method: A Unity method invoked when a collision occurs between two colliders.

- Script Integration: Easily integrates with Unity scripts.
- Object-Specific Events: Executes code specific to the colliding objects.

Example: Executing code to play a sound effect when a player collides with a collectible item.

Trigger Collisions and OnTriggerEnter

Trigger Collisions: Detecting interactions between colliders without physical collision response.

- Area-Based Detection: Ideal for region-specific interactions.
- No Physical Response: Triggers events without physical collision forces.

Example: Activating a door when a player enters a defined trigger area.

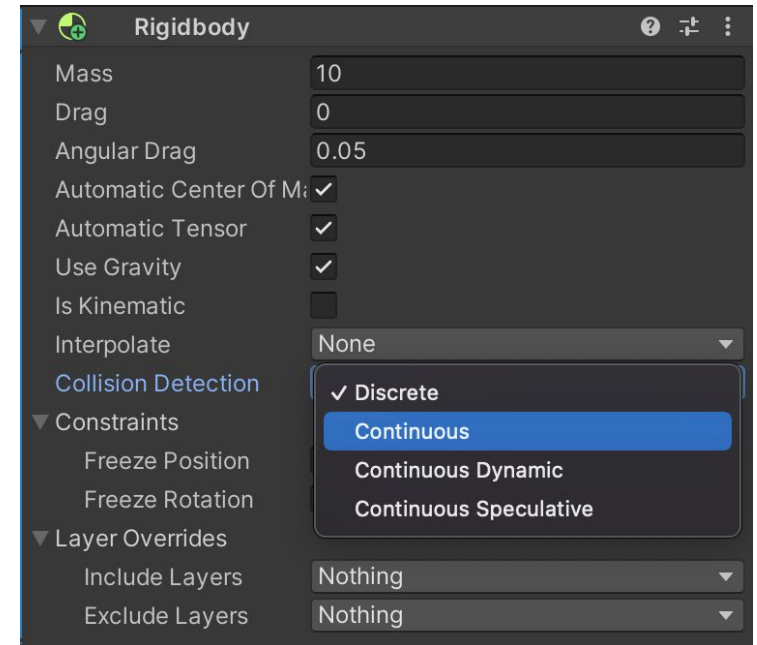
Continuous Collision Detection

Continuous collision detection (CCD) ensures that fast-moving bodies collide with objects instead of passing, or tunnelling, through those objects. Unity provides the following CCD methods:

- **Sweep-based CCD**
- **Speculative CCD**

To use **sweep-based CCD**, select a Rigidbody in the Inspector window and set Collision Detection to **Continuous** or **Continuous Dynamic**.

For **speculative CCD**, set Collision Detection to **Continuous Speculative**.



Sweep-based CCD

Sweep-based CCD: A collision detection technique that checks an object's path between frames to avoid tunneling through other colliders.

- Precise Collision Detection: Ensures accurate collision checks for fast-moving objects.
- Avoids Tunneling: Minimizes the chance of objects passing through each other.

Example: Preventing a fast-moving bullet from passing through a thin wall by checking its path in small increments.

Speculative CCD

Speculative CCD: A technique that predicts potential collisions and resolves them before they occur.

- Preemptive Resolution: Addresses potential collisions before they happen.
- Enhanced Gameplay: Enables smoother interactions by predicting and handling possible collisions.

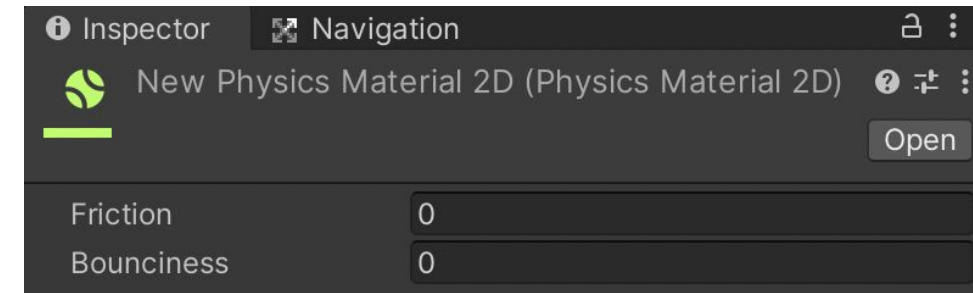
Example: Anticipating a player's character intersecting with an enemy's attack and taking evasive action to avoid the collision.

Physics Material 2D

A **material** is a packet of information that defines the **surface properties** (color, shininess, and so forth) of any object that it's attached to. Defining surface properties separately enables multiple objects to share a material (all the castle walls, for example).

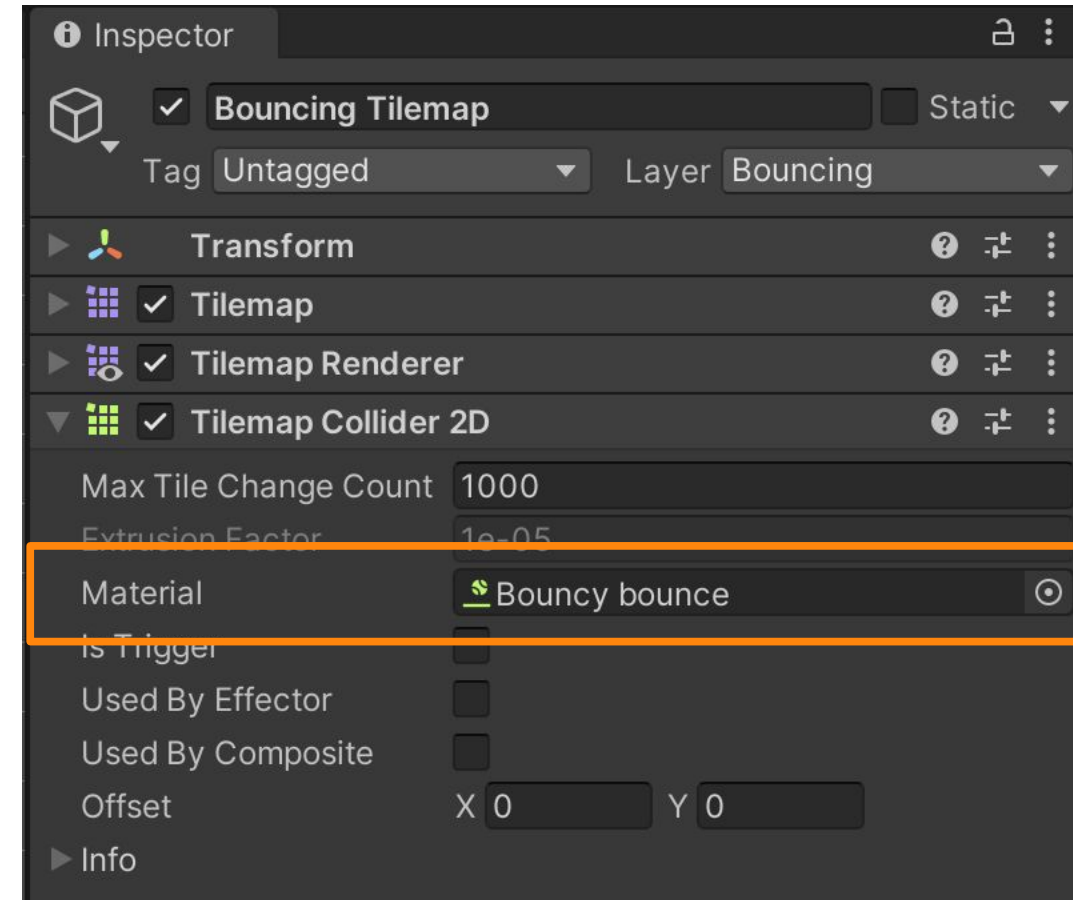
A Physics Material 2D is used to adjust the friction and bounce that occur between 2D physics objects when they collide.

Property:	Function:
Friction	Set the coefficient of friction for this <u>collider</u> .
Bounciness	Set the degree to which <u>collisions</u> rebound from the surface. A value of 0 indicates no bounce while a value of 1 indicates a perfect bounce with no loss of energy.



Physics Material 2D

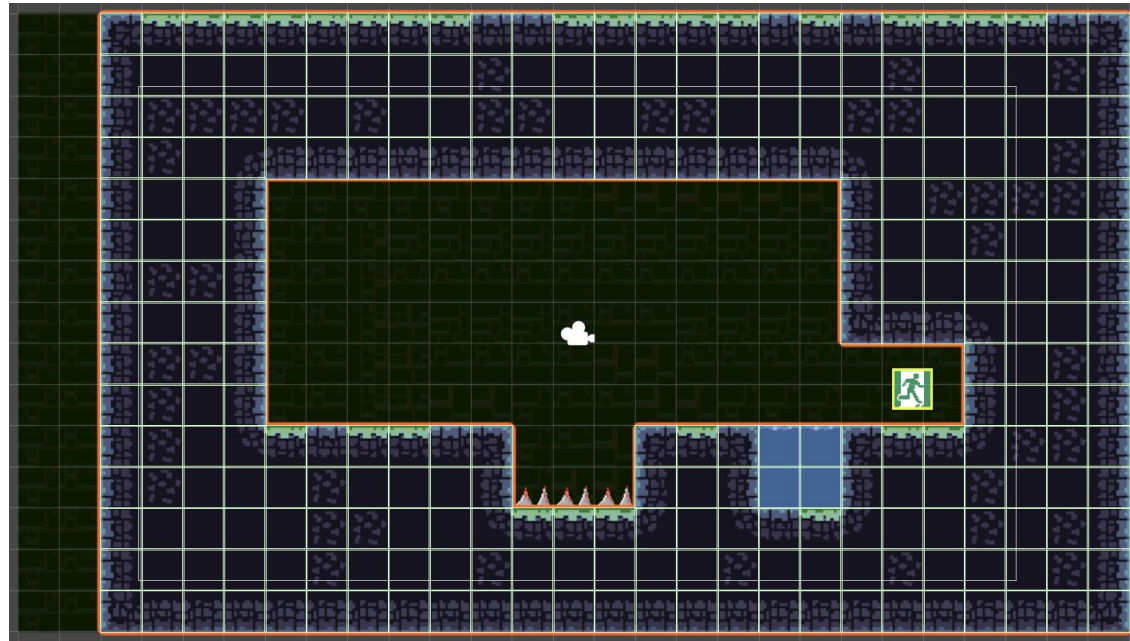
To use your created Physics Material 2D, drag it onto an object with an attached 2D Collider component or drag it onto the collider in the Inspector window.



Practice 1 – Composite Collider

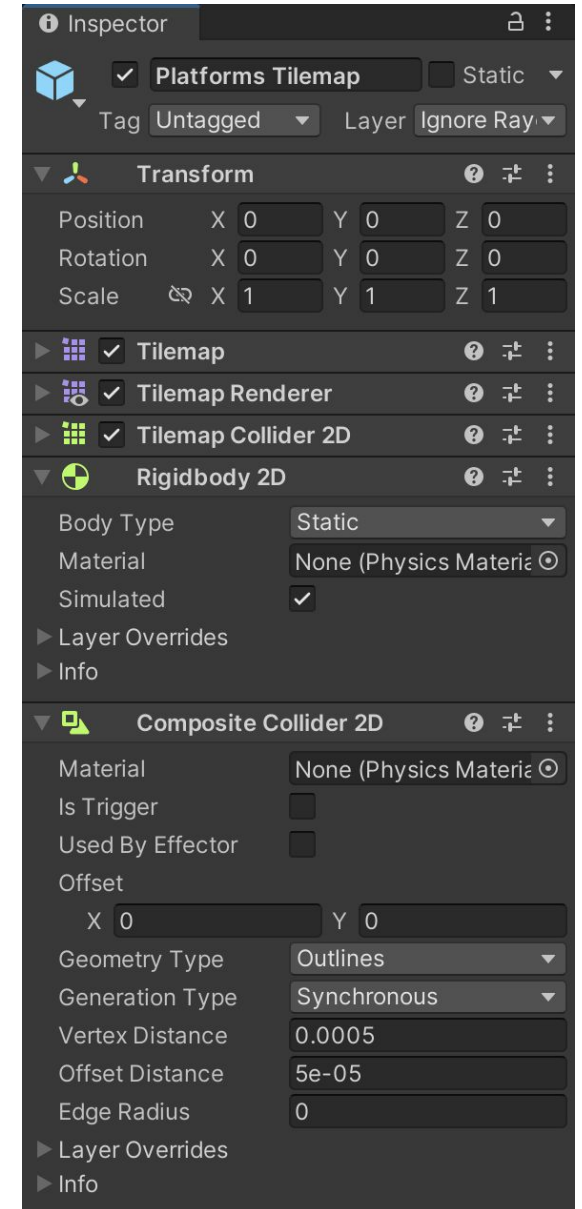
Establish a ground collider to enable the player's interactions, facilitating jumping, falling, and providing a stable surface for the player character to stand upon.

1. Select the Platforms Tilemap GameObject, add a Tilemap Collider 2D component.



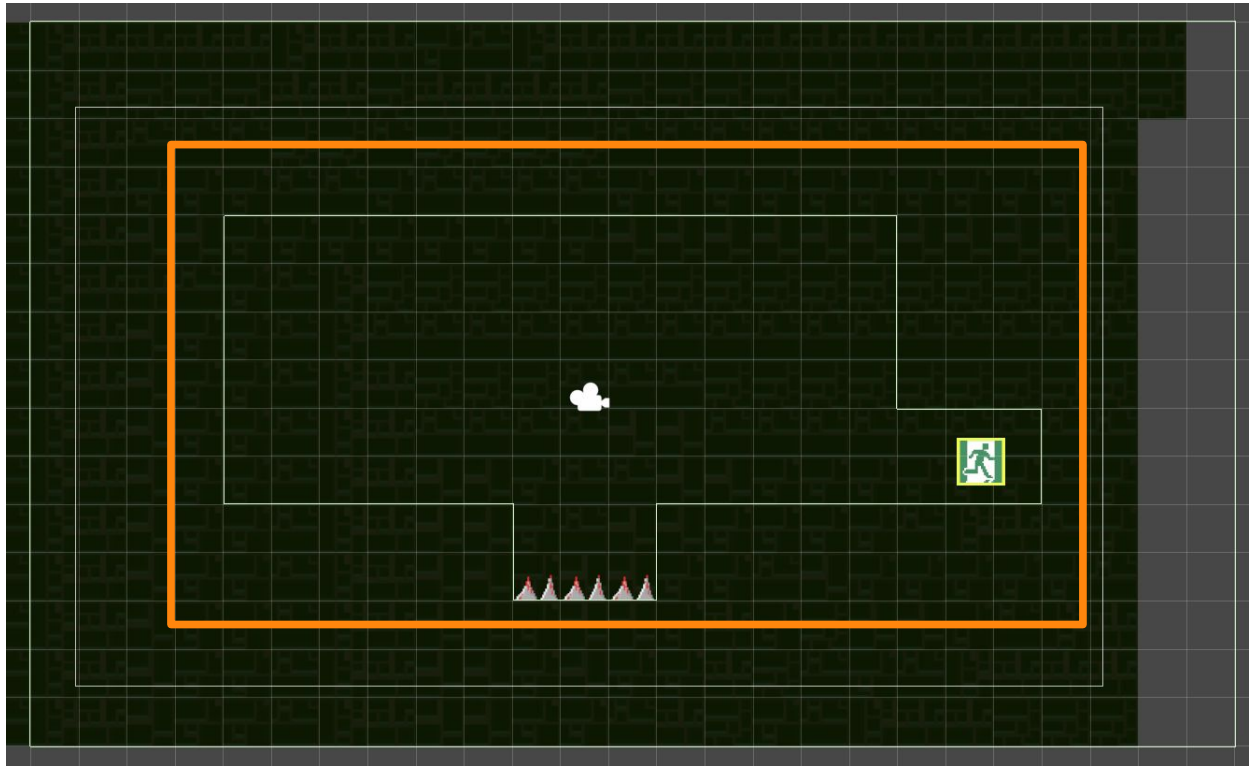
Practice 1 – Composite Collider

2. Add a Composite Collider 2D component. Unity will automatically add a Rigidbody component.
1. In the Rigidbody component, set the body type to static.
2. Within the Tilemap Collider 2D, enable the "Used by Composite" checkbox.



Practice 1 – Composite Collider

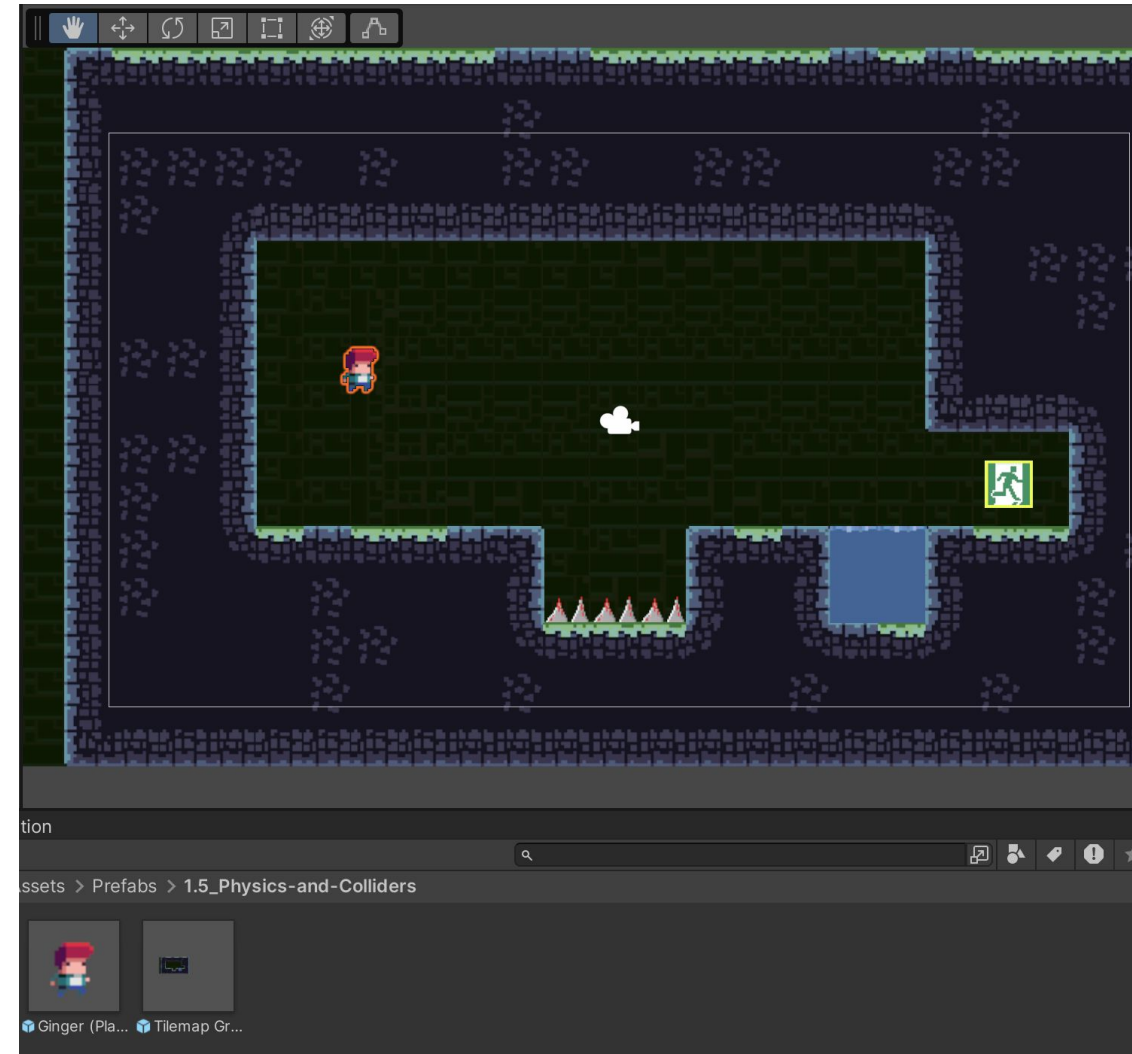
To test, you can disable Tilemap Renderer to see green lines on the Scene Window.



Practice 2 – Add Physics components to Player

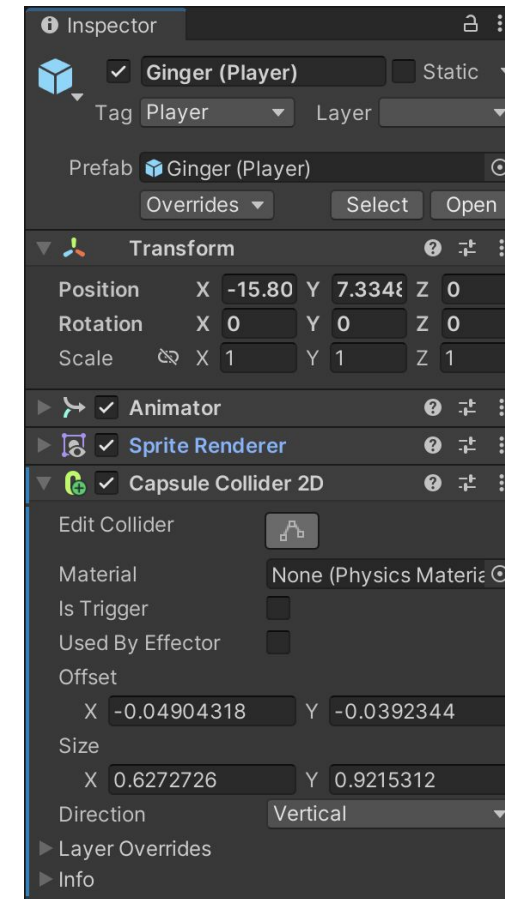
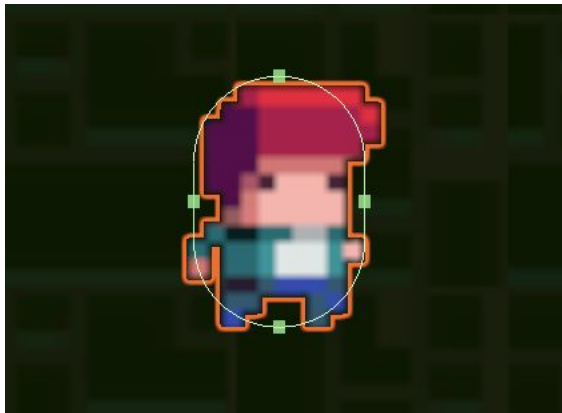
Add components to the Player GameObject to fall and stand on the ground:

1. Go to the Prefabs folder and drag and drop Ginger (the player) onto the scene.



Practice 2 – Add Physics components to Player

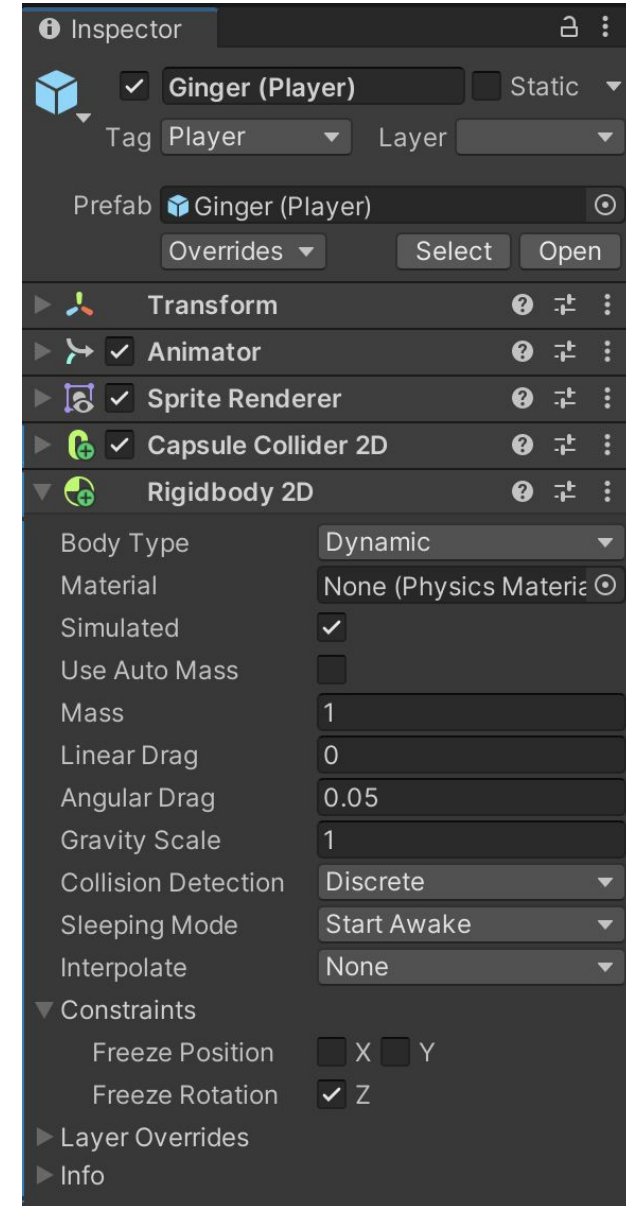
2. Select the Ginger GameObject and add a Capsule Collider 2D component. Click Edit Collider and scale the Capsule Collider.



Practice 2 – Add Physics components to Player

2. Add Rigidbody 2D component to Ginger, and set Body Type is Dynamic.
3. Set Constraints - Free Rotation to true for the Rigidbody 2D component.

Press Play and test the falling and stading movements of Ginger.



Conclusion and Next Steps

- We explored Unity's physics engine and the vital use of colliders in collision detection.
- Understanding the importance of physics in gaming.
- Covered key concepts and practical implementations, providing a foundational understanding for creating engaging experiences while emphasizing best practices in Unity's physics integration.

References

- 1: Hocking, Joseph; Schell, Jesse, Unity in action: multiplatform game development in C#, 2022
- 3: Geig, Mike, Sams teach yourself Unity Game development in 24 hours, 2014
- 6: Gibson Bond, Jeremy, Introduction to Game Design, Prototyping, and Development: From Concept to Playable Game with Unity and C,
- 8: Unity Technologies, Unity Manual, 2023
- 10: Goldstone Will, Unity 3.x game development essentials, 2011